UJIAN TENGAH SEMESTER

PEMROGRAMAN BERORIENTASI OBJEK

Oleh:

Fitri Nur Aisyah

250660221049

Sistem Informasi 3A

BAGIAN A:

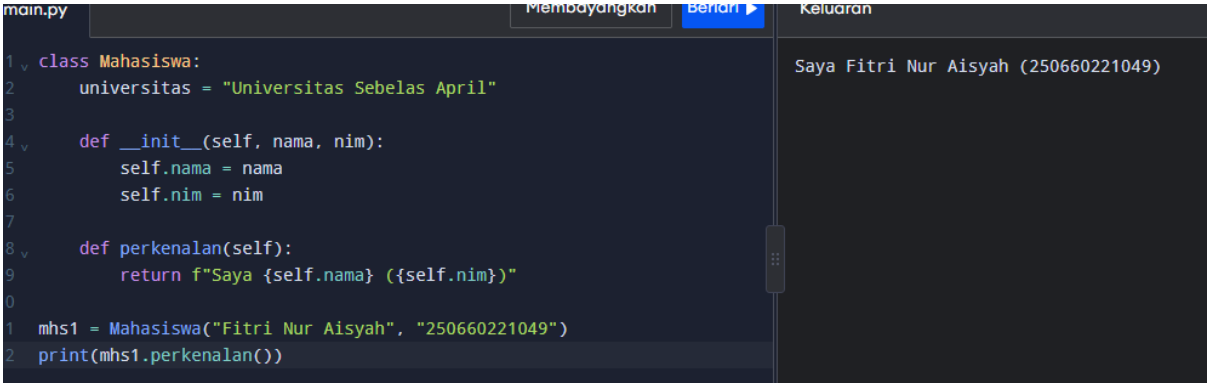
Soal A1:

Jelaskan pengertian dari kelima konsep berikut dan berikan satu contoh penerapannya pada kode Python:

1. class
2. object
3. attribute
4. method
5. constructor

Jawaban:

1) class merupakan cetak biru (blueprint) atau templat yang mendefinisikan struktur dan perilaku sekelompok objek, yaitu attribute apa yang dimilliki dan method apa yang dapat dijalankan.
2) class yang dibuat di memori, memiliki data (*attribute*) dan perilaku (*method*) sesuai class-nya. Satu class dapat menghasilkan banyak object dengan data berbeda.
3) Attributeadalah variabel yang melekat pada class atau object dan menyimpan data/keadaan (*state*) object tersebut. Ada dua jenis:
    - Instance attribute: milik masing-masing object, dibuat dengan self.nama = ...
    - Class attribute: dimiliki bersama oleh semua object dari class.
4) Method adalah fungsi yang didefinisikan di dalam class dan menggambarkan perilaku object. Method instance selalu menerima parameter pertama `self`, yaitu referensi ke object yang memanggilnya, sehingga method dapat mengakses attribute object tersebut.
5) Constructor adalah method khusus yang otomatis dijalankan saat object dibuat, berfungsi menginisialisasi attribute awal object. Di Python, constructor ditulis dengan method `__init__`.
6) Contoh kode yang dipkai dari kelima konsep tersebut:



SOAL A2

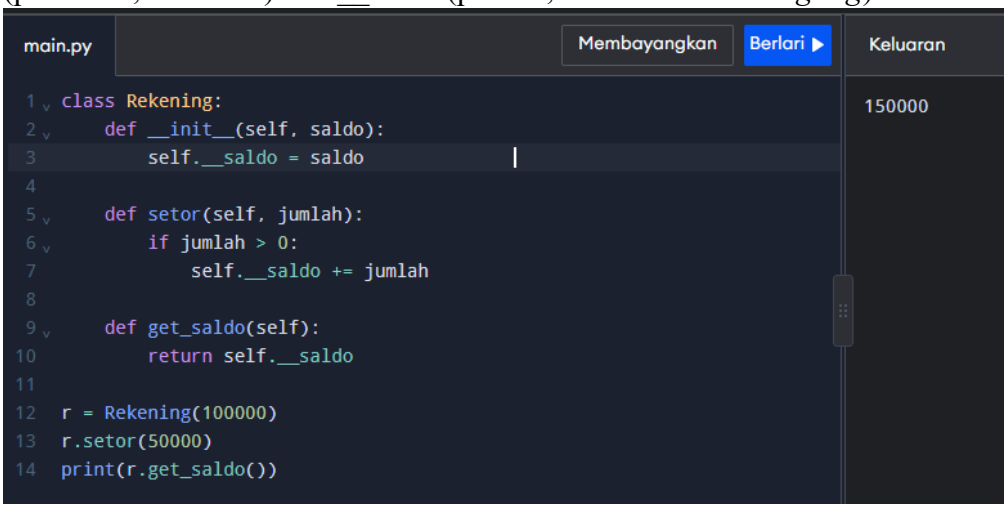
Jelaskan keempat pilar OOP berikut beserta penggunaannya pada program Python:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Untuk setiap pilar, sertakan satu contoh kode singkat yang menunjukkan penerapannya.

Jawaban:

1. Encapsulation adalah penggabungan *attribute* dan *method* dalam satu *class* serta pembatasan akses langsung ke data internal object. Data hanya boleh diubah lewat *method* yang disediakan, sehingga data terlindungi dari perubahan yang tidak valid. Di Python, pembatasan akses dilakukan dengan konvensi penamaan: _nama (protected, konvensi) dan __nama (private, terkena *name mangling*).



2. *Inheritance* (pewarisan) adalah mekanisme ketika sebuah class (*subclass/child*) mewarisi *attribute* dan *method* dari class lain (*superclass/parent*). Tujuannya agar kode dapat dipakai ulang (*reusability*) dan hubungan “is-a” dapat dimodelkan.

main.py | Membayangkan | Berlari ▶ | Keluaran

```python
class Hewan:
    def __init__(self, nama):
        self.nama = nama

    def makan(self):
        return f"{self.nama} sedang makan"

class Kucing(Hewan):
    def mengeong(self):
        return f"{self.nama} mengeong"

k = Kucing("Tom")
print(k.makan())
print(k.mengeong())
```

```
Tom sedang makan
Tom mengeong
```

3. *Polymorphism* (“banyak bentuk”) adalah kemampuan *method* dengan nama yang sama berperilaku berbeda tergantung *object* yang memanggilnya. Bentuk yang umum adalah *method overriding*, yaitu subclass menimpa method milik superclass.

main.py | Membayangkan | Berlari ▶ | Keluaran

```python
class Hewan:
    def suara(self):
        return "..."

class Kucing(Hewan):
    def suara(self):
        return "Meong"

class Anjing(Hewan):
    def suara(self):
        return "Guk guk"

for h in [Kucing(), Anjing()]:
    print(h.suara())
```

```
Meong
Guk guk
```

4. Abstraction adalah penyembunyian detail implementasi dan hanya menampilkan fungsi yang esensial. Di Python, abstraction dibuat dengan *abstract class* (modul abc) yang berisi *abstract method*: method tanpa isi yang wajib di-override oleh subclass. Abstract class tidak dapat dibuat object-nya.

main.py | Membayangkan | Berlari ▶ | Keluaran

```python
from abc import ABC, abstractmethod

class Bentuk(ABC):
    @abstractmethod
    def luas(self):
        pass

class Persegi(Bentuk):
    def __init__(self, sisi):
        self.sisi = sisi

    def luas(self):
        return self.sisi ** 2

p = Persegi(4)
print(p.luas())
```

```
16
```

SOAL A3:
Perhatikan rancangan class berikut:

```
class Pengguna:
    def __init__(self, nama, email):
        self.__nama = nama
        self.__email = email

    @property
    def nama(self):
        return self.__nama

class Mahasiswa(Pengguna):
    def __init__(self, nama, email, nim):
        super().__init__(nama, email)
        self.nim = nim

    def tampilkan_peran(self):
        return f"{self.nama} adalah mahasiswa dengan NIM {self.nim}"
```

Jawab pertanyaan berikut:

1. Konsep OOP mana yang diterapkan pada self.__nama dan @property? Jelaskan alasan teknisnya.
2. Konsep OOP mana yang diterapkan pada class Mahasiswa(Pengguna)? Jelaskan apa yang dilakukan super().
3. Mengapa self.nama dapat dipanggil pada method tampilkan_peran() di subclass meskipun attribute tersebut privat di superclass?

Jawaban:

1. Konsep pada self.__nama dan @property

Konsep: Encapsulation, karena:

- self.__nama (diawali dua underscore) adalah *private attribute*. Python menerapkan *name mangling*, sehingga di luar class nama ini berubah menjadi _Pengguna__nama dan tidak bisa diakses langsung lewat obj.__nama.
- @property mengubah method nama() menjadi *read-only attribute*. Data dibaca lewat obj.nama (tanpa tanda kurung), tetapi karena tidak ada *setter*, nilainya tidak dapat diubah dari luar (obj.nama = "X" memunculkan AttributeError).
- Hasilnya: data internal dilindungi, dan akses dikendalikan lewat antarmuka yang disediakan class.

2. Konsep pada class Mahasiswa(Pengguna)

Konsep: Inheritance (pewarisan). Mahasiswa adalah *subclass* (child) dari Pengguna sebagai *superclass* (parent), sehingga mewarisi *attribute* dan *method* Pengguna (hubungan *is-a*: mahasiswa adalah pengguna).

- Fungsi super(): super() mengembalikan referensi ke superclass sehingga subclass dapat memanggil method milik parent. Pada super().__init__(nama, email), constructor Pengguna dijalankan terlebih dahulu untuk menginisialisasi __nama dan

__email. Setelah itu, constructor Mahasiswa menambahkan attribute miliknya sendiri, yaitu nim.

Tanpa super().__init__(...), attribute __nama dan __email tidak pernah dibuat, sehingga pemanggilan self.nama akan menimbulkan error.

3. self.nama bisa dipanggil di subclass karena yang dipanggil di tampilkan_peran() adalah self.nama (property publik), bukan self.__nama (attribute privat).
   Alurnya:
   Mahasiswa mewarisi property nama dari Pengguna (property bersifat publik). Saat self.nama dipanggil, Python menjalankan method nama() milik Pengguna. Method itu berada di dalam class Pengguna, sehingga self.__nama di dalamnya otomatis diubah menjadi self._Pengguna__nama, dan atribut itu memang ada pada object (dibuat oleh super().__init__).
   Jadi subclass tidak mengakses attribute privat secara langsung, melainkan lewat antarmuka publik yang disediakan superclass. Inilah tujuan encapsulation.

BAGIAN B

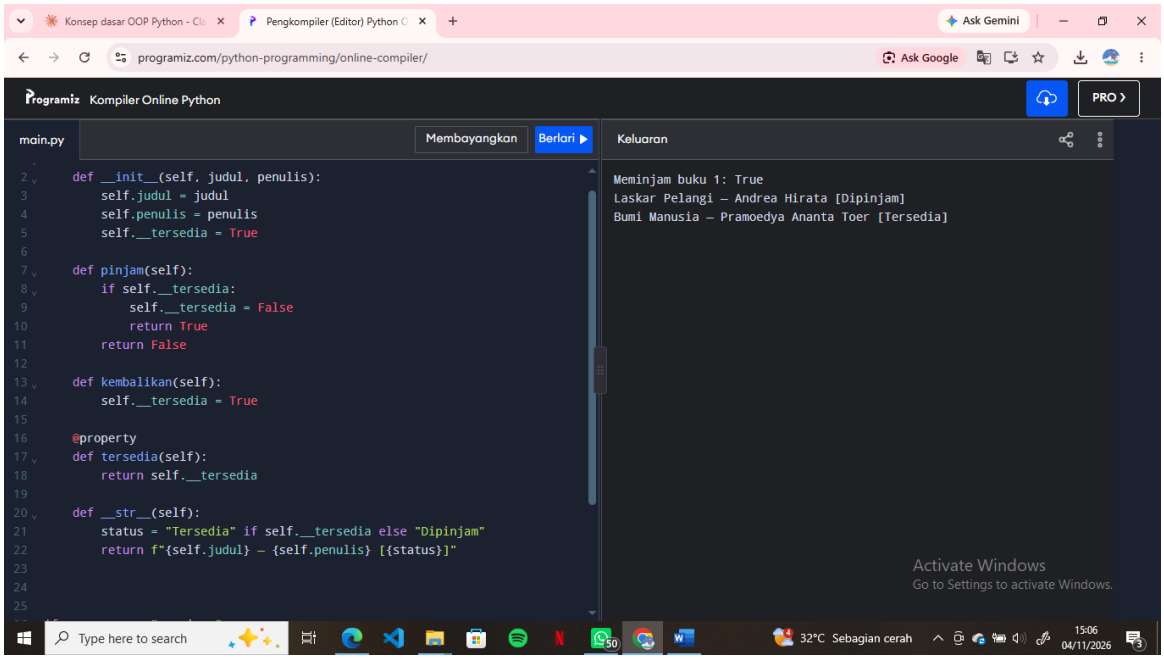
SOAL B1:

Buat class Buku dengan ketentuan berikut:

1. Constructor menerima judul (str) dan penulis (str).
2. Attribute tersedia diinisialisasi dengan True dan bersifat privat.
3. Method pinjam() mengubah tersedia menjadi False dan mengembalikan True jika buku tersedia. Jika tidak tersedia, kembalikan False.
4. Method kembalikan() mengubah tersedia menjadi True.
5. Property tersedia mengembalikan status ketersediaan tanpa bisa diubah langsung dari luar class.
6. Method __str__() mengembalikan string berformat: "judul — penulis [Tersedia]" atau "judul — penulis [Dipinjam]".

Tulis program utama yang membuat dua object Buku, meminjam satu buku, dan menampilkan status kedua buku.

Jawaban:



Penjelasan:

Constructor menyimpan judul dan penulis, serta membuat attribute privat __tersedia bernilai True. Method pinjam() mengubah __tersedia menjadi False dan mengembalikan True jika buku tersedia, selain itu mengembalikan False. Method kembalikan() mengubahnya kembali menjadi True.

tersedia hanya bisa dibaca, sehingga mengubahnya dari luar menimbulkan AttributeError. Method __str__() menampilkan status “Tersedia” atau “Dipinjam” sesuai nilai __tersedia. Program utama membuat dua object, meminjam buku1, lalu mencetak keduanya.

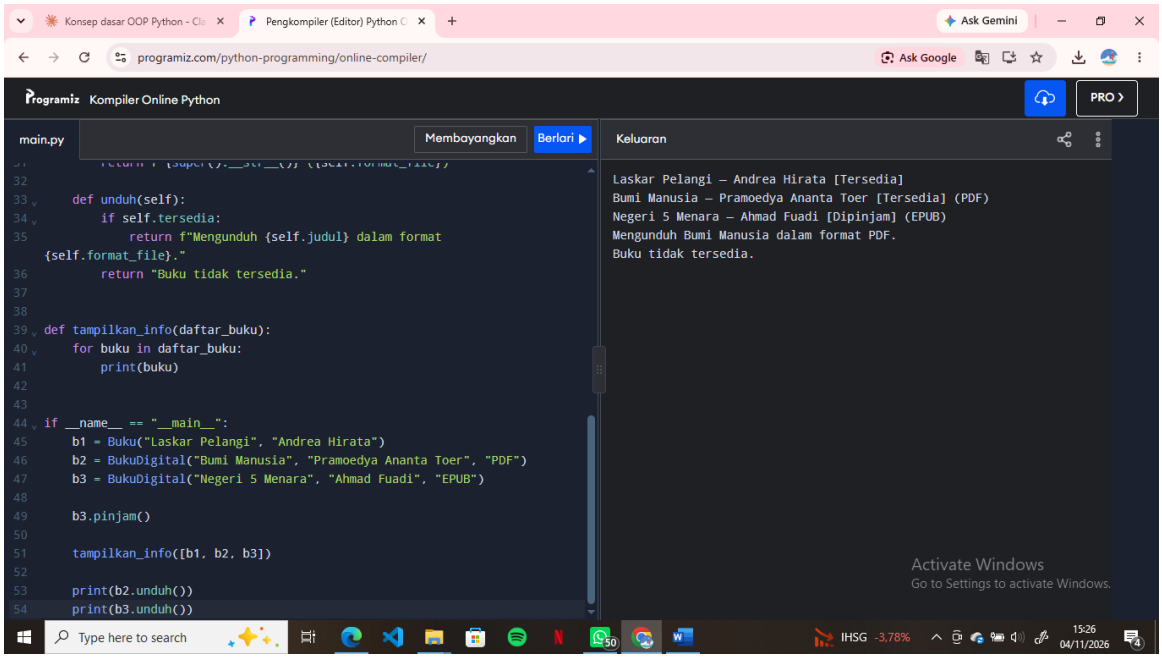
Soal B2:

Diberikan class Buku dari Soal B1. Buat subclass BukuDigital dengan ketentuan berikut:

1. Subclass mewarisi Buku dan menambahkan attribute format_file (str, misalnya "PDF" atau "EPUB").
2. Constructor menggunakan super() untuk memanggil constructor parent.
3. Override method __str__() sehingga formatnya menjadi: "judul — penulis [Dipinjam] (format_file)".
4. Method unduh() mengembalikan string "Mengunduh judul dalam format format_file." jika buku tersedia, dan "Buku tidak tersedia." jika sedang dipinjam.

Selanjutnya, tulis fungsi tampilkan_info(daftar_buku) yang menerima list berisi object Buku dan BukuDigital, lalu mencetak __str__() dari setiap object. Fungsi ini harus bekerja untuk kedua tipe object tanpa pengecekan tipe eksplisit.

Jawaban:



BukuDigital(Buku) mewarisi semua attribute dan method Buku (*inheritance*), lalu menambahkan format_file. Constructor-nya memanggil super().__init__(judul, penulis) agar judul, penulis, dan __tersedia tetap diinisialisasi oleh parent.

Method __str__() di-*override*, tetapi memanggil super().__str__() untuk memakai format parent, lalu menambahkan (format_file) di belakangnya. Status [Tersedia] atau [Dipinjam] tetap menyesuaikan kondisi buku (soal hanya mencontohkan [Dipinjam]).

Method unduh() membaca status lewat property self.tersedia, bukan self.__tersedia. Ini penting karena __tersedia privat milik Buku; di dalam BukuDigital, nama itu akan diubah menjadi _BukuDigital__tersedia dan menimbulkan AttributeError.

Fungsi tampilkan_info() hanya memanggil print(buku) untuk setiap elemen, yang otomatis memanggil __str__(). Python memilih versi __str__() sesuai tipe object sebenarnya, sehingga fungsi bekerja untuk Buku maupun BukuDigital tanpa isinstance(). Inilah penerapan polymorphism.

Soal B3:

Perhatikan kode berikut yang memiliki beberapa kesalahan:

```
class RekeningBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.saldo = saldo
    def setor(self, jumlah):
        self.saldo = self.saldo + jumlah
```

```
    def tarik(self, jumlah):
        self.saldo = self.saldo - jumlah
    def tampilkan_saldo(self):
        print(f"Saldo {self.pemilik}: Rp{self.saldo}")
```

Tugas:

1. Identifikasi tiga kesalahan atau kelemahan pada kode tersebut dari perspektif OOP.
2. Perbaiki kode dengan menerapkan encapsulation (attribute privat), validasi pada method setor() dan tarik() (jumlah harus positif dan penarikan tidak boleh melebihi saldo), serta property untuk mengakses saldo.
3. Tulis program utama yang mendemonstrasikan: setor uang, tarik uang yang valid, tarik uang yang melebihi saldo (harus menolak), dan tarik jumlah negatif (harus menolak).

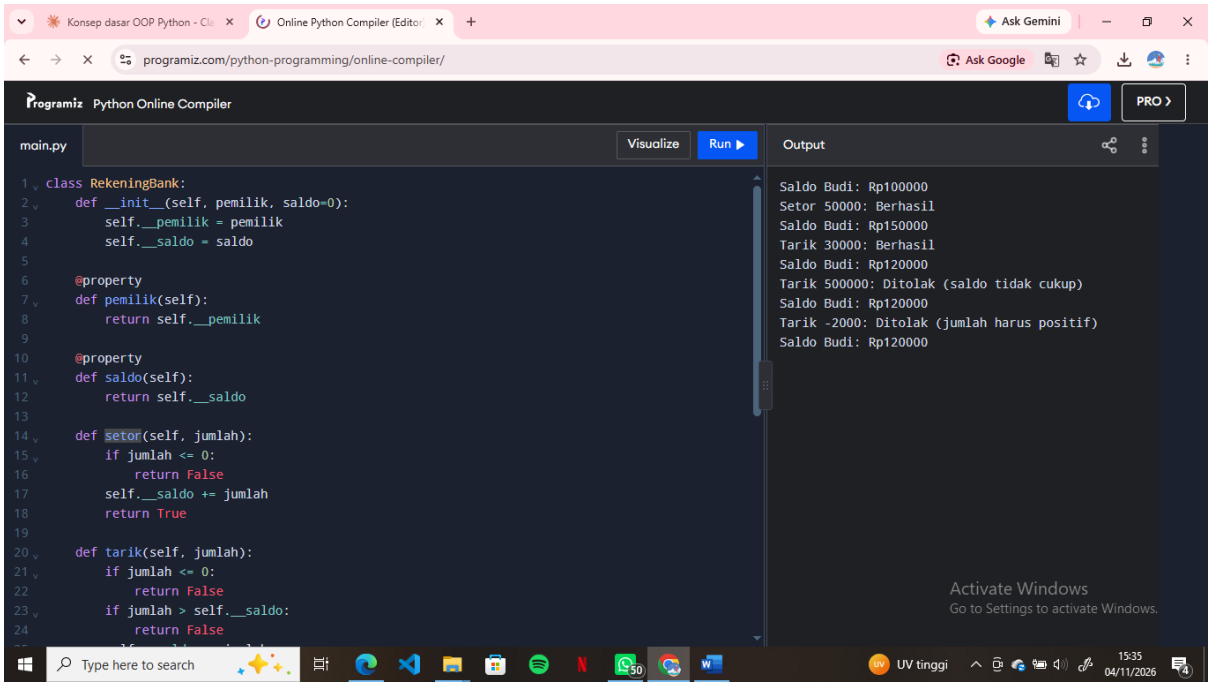
Jawaban:

1. Tiga kesalahan/kelemahan (perspektif OOP)

1. **Tidak ada encapsulation.** self.saldo dan self.pemilik bersifat publik, sehingga bisa diubah langsung dari luar, misalnya rek.saldo = 999999999, tanpa melewati aturan apa pun.
2. **Tidak ada validasi pada setor() dan tarik().** Jumlah negatif atau nol diterima. setor(-500) justru mengurangi saldo, dan tarik(-500) menambah saldo.
3. **Penarikan boleh melebihi saldo.** tarik() tidak memeriksa saldo, sehingga saldo bisa menjadi negatif. Selain itu, tidak ada umpan balik keberhasilan atau kegagalan (method tidak mengembalikan nilai), dan tidak ada property untuk membaca saldo secara terkendali.

2. kode perbaikkan dan

3. program utama



Penjelasan

Attribute __pemilik dan __saldo dibuat privat, sehingga tidak bisa diubah langsung dari luar class. Property saldo hanya punya getter, jadi rek.saldo = 1000000 menimbulkan AttributeError dan saldo hanya bisa berubah lewat setor() dan tarik().

Method setor() menolak jumlah nol atau negatif. Method tarik() menolak jumlah nol atau negatif, dan juga menolak jika jumlah melebihi saldo. Kedua method mengembalikan True jika berhasil dan False jika ditolak, sehingga program pemanggil bisa tahu hasilnya.

.